

Using Reusable Development Skills

A practical guide to all 14 context-aware AI workflows

Java, Python and C# are end-to-end. All other profiles are standards-only scaffolds.

AI Engineering Starter Kit | Current edition | July 2026

Contents

Section	Purpose
01	Operating model and maturity
02	Context and installation
03	The 14-skill catalogue
04	Java skills
05	Python skills
06	C# / .NET skills
07	Worked feature workflow
08	Testing workflow
09	Review and security
10	Tool selection
11	Human-only Git
12	Troubleshooting and quick reference

Operating model and honest maturity

Context defines what is true. Skills define how work is performed. Gates provide evidence. Humans publish and approve.

Layer	Responsibility
Repository context	Version-controlled project, architecture, coding, testing, security and dependency rules
Reusable skill	Ordered task procedure that reads repository context before acting
Automated gates	Formatting, analysis, tests and security evidence
Human	Requirement, risk acceptance, staging, commit, push and merge

Profile maturity	Stacks
End-to-end runnable	Java / Spring Boot; Python / FastAPI; C# / ASP.NET Core
Standards-only scaffold	C; C++; Rust; Scala; Node; JavaScript; TypeScript; React; Next.js

Context and installation

- Work inside the target repository so the skill can locate AGENTS.md.
- Install the complete skill folder through the organisation's approved distribution process.
- Every skill reads applicable active .ai/*.md files completely.
- Project facts remain in repository context, not duplicated inside reusable skill procedures.

Invocation

Use \$create-python-feature to add a product search endpoint.
Acceptance criteria: validate input, return 404 when absent, add pytest coverage.
Do not add dependencies.

The 14-skill catalogue

Stack	Count	Skills
Java	6	create-spring-feature; generate-java-tests; review-java-change; diagnose-java-failure; refactor-java-code; secure-java-review
Python	4	create-python-feature; generate-python-tests; review-python-change; secure-python-review
C# / .NET	4	create-dotnet-feature; generate-dotnet-tests; review-dotnet-change; secure-dotnet-review

Scaffold profiles do not yet have bundled stack-specific skills. Use active context and generic prompts until a governed skill set exists.

Java / Spring Boot skills

Skill	Use
create-spring-feature	Implement a complete Spring Boot vertical slice
generate-java-tests	JUnit 5, Mockito, AssertJ and Testcontainers coverage
review-java-change	Independent correctness, architecture and compatibility review
diagnose-java-failure	Evidence-first build, test, CI or runtime diagnosis
refactor-java-code	Behaviour-preserving readability and boundary improvements
secure-java-review	Threat-focused review of Java code and configuration

Example

Use \$create-spring-feature to add a product lookup endpoint.
Use approved Spring libraries, update OpenAPI and run mvn verify.

Python / FastAPI skills

Skill	Use
create-python-feature	Implement a typed FastAPI feature with explicit dependencies
generate-python-tests	pytest coverage with meaningful outcome assertions
review-python-change	Review typing, state, correctness and maintainability
secure-python-review	Review validation, injection, secrets and dependency risk

- Use strict Pydantic v2 models at boundaries.
- Constrain mutable module-level state.
- Prefer application factories, fresh fixtures and explicit dependency injection.
- Run Ruff format/lint, strict mypy and pytest.

C# / .NET skills

Skill	Use
create-dotnet-feature	Implement an ASP.NET Core feature with clear boundaries
generate-dotnet-tests	xUnit, NSubstitute and FluentAssertions coverage
review-dotnet-change	Review nullability, contracts, async use and architecture
secure-dotnet-review	Review authorization, validation, logging and dependency risk

Example

Use \$create-dotnet-feature to add a product lookup endpoint.
Preserve nullable contracts, add xUnit tests and run the .NET verification baseline.

Worked feature workflow

Stage	Invocation	Human decision
Orient	Skill loads AGENTS.md and .ai context	Confirm scope and acceptance criteria
Implement	create-*-feature	Inspect design and dependency choices
Test	generate-*-tests	Confirm meaningful behaviours
Review	review-*-change	Resolve blockers
Secure	secure-*-review	Accept or remediate risk
Verify	Stack-specific gates	Confirm evidence
Publish	No skill	Human stages, commits, pushes and opens PR

Testing workflow

Every test must contain at least one meaningful assertion about an observable result.

- Assert a returned value, state transition, HTTP contract, persisted result or exception.
- Interaction verification alone is insufficient.
- Mock only external, slow or non-deterministic collaborators.
- Use real integration boundaries when the risk warrants them.
- Treat excessive mock setup as a design warning.

Use `$generate-java-tests` to cover the product service failure paths.
Include positive, missing-record and invalid-input cases.
Do not change production behaviour without reporting the defect first.

Review and security workflows

Review

Use `$review-dotnet-change` to review the current diff.
Do not edit files. Cite exact files and lines and order findings by severity.

Security

Use `$secure-python-review` to assess the current API change.
Review authorization, input validation, secrets, logging and abuse cases.

- Review skills remain independent from implementation.
- Findings explain a credible failure scenario and impact.
- High-risk ambiguity is escalated to an accountable owner.

Selecting a library or tool

- Read `.ai/tooling-policy.md`, active dependencies context and manifests.
- Use the approved choice when one exists.
- When no choice is documented, stop and ask before adding anything.
- Explain alternatives, licence, provenance, security, compatibility and maintenance cost.
- Ask whether approval is task-local or an ongoing project standard.
- For an ongoing standard, update context, manifest, lockfile, CI and hooks together.

Human-only Git

AI prepares work; accountable people publish it.

- Never stage or commit.
- Never push or force-push.
- Never create or switch branches.
- Never amend, tag, merge, rebase, cherry-pick or reset history.
- Never open, approve or merge a pull request.
- Never delegate those actions to another agent or tool.
- Leave a handoff containing changed files, evidence, risks and an optional commit-message suggestion.

Troubleshooting and quick reference

Problem	Response
Skill not recognised	Install the complete folder and refresh the AI session
Context not found	Open the repository root containing AGENTS.md
Dependency unspecified	Stop and request an explicit selection
Verification unavailable	Report the gap; do not claim a pass
Scaffold profile selected	Add product-owned template, tests, CI and skills before delivery

Create: Use `$create-[stack]-feature` to implement [outcome].
Test: Use `$generate-[stack]-tests` to prove [behaviours].
Review: Use `$review-[stack]-change`; review only, do not edit.
Secure: Use `$secure-[stack]-review`; return severity-ranked findings.